

# FILESYSTEM

## Evolución Histórica y Diseños Clásicos de los Sistemas de Archivos

Desde una perspectiva histórica, los sistemas de archivos han evolucionado desde diseños FAT (File Allocation Table), que utilizaba tablas enlazadas para gestionar clústeres en discos de baja capacidad, hasta sistemas como NTFS, ext4 o ZFS, que incorporan estructuras de metadatos avanzadas (inodos, B-trees), técnicas de journaling y algoritmos de checksum para garantizar integridad. Esta evolución no es meramente técnica, sino una respuesta a necesidades críticas: escalabilidad para manejar petabytes de datos, resistencia a la corrupción en entornos de alta concurrencia y adaptación a dispositivos como SSDs, donde la escritura secuencial ya no es el paradigma dominante.

### 1. Origen y Evolución de FAT

#### FAT12, FAT16 y FAT32 (Microsoft, 2000)

El sistema de archivos FAT (File Allocation Table) fue uno de los primeros en ser ampliamente utilizado en computadoras personales. Su evolución, desde FAT12 hasta FAT32, refleja los avances en el tamaño y la capacidad de almacenamiento, así como las mejoras en la eficiencia de los sistemas de gestión de archivos.

**FAT12** fue utilizado en los primeros sistemas de archivos para discos duros de pequeña capacidad. Su principal limitación es el tamaño máximo de partición de 32 MB.

**FAT16** permitió el soporte de discos más grandes, con un tamaño máximo de partición de 2 GB, pero seguía siendo limitado en comparación con sistemas posteriores.

**FAT32** resolvió muchos de los problemas anteriores, con un tamaño máximo de partición de 2 TB y un tamaño de archivo de hasta 4 GB. Este sistema es ampliamente utilizado en dispositivos de almacenamiento externo como unidades USB y tarjetas SD.

Tabla de Evolución de Sistemas de Archivos FAT y NTFS

| Sistema de Archivos | Año de Introducción | Capacidad Máxima de Archivo | Tamaño Máximo Volumen | Tamaño de Bloque | Journaling | Sistema de Asignación | Compatibilidad Versiones Anteriores |
|---------------------|---------------------|-----------------------------|-----------------------|------------------|------------|-----------------------|-------------------------------------|
| <b>FAT12</b>        | 1977                | 32 MB                       | 16 MB                 | 512 B – 4 KB     | No         | Asignación            | No                                  |
| <b>FAT16</b>        | 1984                | 2 GB                        | 4 GB                  | 512 B – 64 KB    | No         | Asignación            | No                                  |
| <b>FAT32</b>        | 1996                | 4 GB                        | 2 TB                  | 512 B – 32 KB    | No         | Asignación            | No                                  |
| <b>exFAT</b>        | 2006                | 16 EB                       | 128 PB                | 4 KB – 64 KB     | No         | Asignación            | No                                  |
| <b>NTFS</b>         | 1993                | 16 EB                       | 256 TB                | 512 B – 64 KB    | Sí         | B-Trees               | Sí                                  |
| <b>ReFS</b>         | 2012                | 35 PB                       | 1 YB                  | 4 KB – 64 KB     | Sí         | B-Trees               | No                                  |

#### 1.1. Funcionamiento Interno de FAT

**Tabla de Asignación de Archivos (FAT):** La tabla FAT es la estructura central en la que se almacena la ubicación de cada bloque de un archivo. Cada entrada en la tabla FAT indica si el bloque está libre, ocupado o dañado.

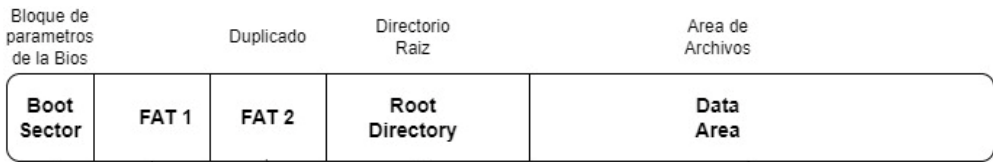
**Clusters, Sectores y Tamaño de Entrada:** Los archivos se dividen en clusters, que son unidades de almacenamiento contiguas. El tamaño de un cluster depende del tamaño de la partición, y la entrada de la tabla FAT apunta a la ubicación del siguiente cluster del archivo.

**Fragmentación Interna y Externa:** La fragmentación interna ocurre cuando los clusters no se utilizan por completo. La fragmentación externa se refiere a la dispersión de los clusters a lo largo del disco, lo que reduce la eficiencia de acceso.

**Estrategias de Recuperación de Archivos:** La recuperación en sistemas FAT se realiza buscando entradas de la tabla FAT que apunten a bloques perdidos o dañados, dependiendo de la situación.

1.2. Estructura del Sistema de Archivo

Existen cinco componentes principales en la estructura del sistema de archivos FAT. Sin embargo, su descripción puede variar dependiendo del nivel de detalle. Esta variación surge de la necesidad de distinguir entre la estructura física del disco y la estructura lógica del sistema de archivos:



MBR (Master Boot Record)

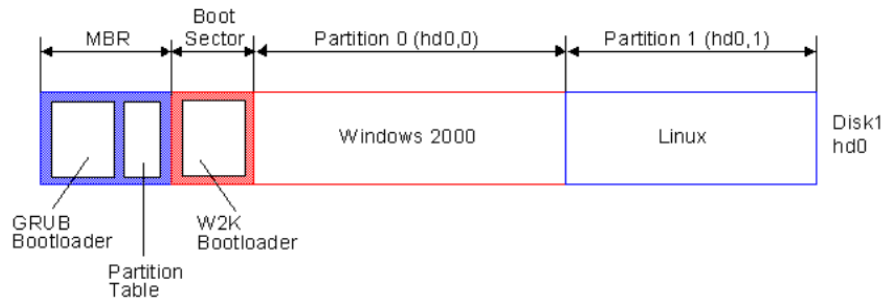
El MBR se encuentra en el *primer sector físico* del disco (*sector 0*), constituyendo el sector de arranque primario. Contiene información sobre la configuración de las particiones y el código de arranque del sistema. Este sector también incluye el código de arranque que el BIOS ejecuta para iniciar el sistema operativo.

El MBR tiene un tamaño fijo de 512 bytes y está estructurado de la siguiente manera:

| Offset | Tamaño    | Descripción                         |
|--------|-----------|-------------------------------------|
| 0x000  | 446 bytes | Código de arranque (Bootstrap Code) |
| 0x1BE  | 16 bytes  | Entrada de partición 1              |
| 0x1CE  | 16 bytes  | Entrada de partición 2              |
| 0x1DE  | 16 bytes  | Entrada de partición 3              |
| 0x1EE  | 16 bytes  | Entrada de partición 4              |
| 0x1FE  | 2 bytes   | Firma del MBR (0x55AA)              |

El MBR puede contener información de hasta *cuatro* particiones primarias. Si un disco está particionado en más de una partición, cada partición tendrá su propio *Boot Sector* y sistema FAT independiente. Si el MBR está dañado, el sistema puede volverse inarrancable, y se requerirá una herramienta de recuperación como *TestDisk* para restaurarlo.

Si un disco de *1 TB* se divide en tres particiones de 300 GB, 300 GB y 400 GB, cada partición tendrá su propia estructura FAT independiente. El MBR solo mantendrá la información de las particiones, mientras que cada Boot Sector administrará el sistema de archivos dentro de su partición correspondiente.



Esta imagen tenemos un disco duro (Disk1/hd0) con dos particiones. El MBR, ubicado en el primer sector del disco, es único y contiene el cargador de arranque GRUB y la tabla de particiones. Cada partición tiene su propio Boot Sector independiente: el de la primera partición

(Partition 0/hd0,0) contiene el cargador de arranque de Windows 2000 (W2K Bootloader), y el de la segunda partición (Partition 1/hd0,1) contiene la información necesaria para arrancar Linux.

En una estructura de una entrada de partición (16 bytes) cada entrada en la tabla de particiones sigue este formato:

| Offset | Tamaño  | Descripción                            |
|--------|---------|----------------------------------------|
| 0x00   | 1 byte  | Estado de la partición (0x80 = activa) |
| 0x01   | 3 bytes | Dirección CHS de inicio                |
| 0x04   | 1 byte  | Tipo de partición (0x01 = FAT12)       |
| 0x05   | 3 bytes | Dirección CHS de fin                   |
| 0x08   | 4 bytes | LBA de inicio de la partición          |
| 0x0C   | 4 bytes | Tamaño de la partición en sectores     |

Tipo de sistema de archivos FAT:

- 0x01 → FAT12
- 0x04 → FAT16
- 0x0B → FAT32

## Boot Sector

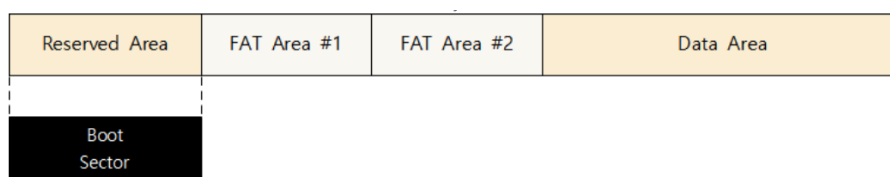
El Boot Sector, a diferencia del MBR, se localiza en el primer sector lógico de cada partición (LBA inicial) y contiene información para la configuración y operación del sistema FAT.

FAT12/16 tiene una estructura de Boot Sector más simple y compacta. Mientras que FAT32 incluye campos adicionales para manejar volúmenes más grandes y proporcionar información adicional para discos duros modernos. El campo "Número de sectores por FAT" (0x24) es fundamental para FAT32

| Offset | Tamaño | Descripción (FAT12/16)                   | Descripción (FAT32)                             |
|--------|--------|------------------------------------------|-------------------------------------------------|
| 0x00   | 3 B    | Salto al código de arranque              | Código de arranque (salto al código ejecutable) |
| 0x03   | 8 B    | Nombre del sistema (p. ej., "MSDOS5.0")  | Nombre del sistema de archivos                  |
| 0x0B   | 2 B    | Tamaño de sector (normalmente 512)       | Bytes por sector                                |
| 0x0D   | 1 B    | Sectores por clúster                     | Sectores por clúster                            |
| 0x0E   | 2 B    | Tamaño de la FAT (en sectores)           | Número de sectores reservados                   |
| 0x16   | 2 B    | Número de entradas en el directorio raíz | Número de copias de FAT                         |
| 0x1C   | 4 B    | Número total de sectores                 | Tamaño total de volumen                         |
| 0x24   | 4 B    | -                                        | Número de sectores por FAT                      |
| 0x2A   | 11 B   | Etiqueta de volumen                      | -                                               |
| 0x2C   | 2 B    | -                                        | Número de sectores por pista                    |
| 0x30   | 2 B    | -                                        | Número de cabezas                               |
| 0x36   | 8 B    | Sistema de archivos ("FAT12 ")           | Nombre del volumen                              |
| 0x3E   | 2 B    | Firma (0x55AA)                           | Sistema de archivos (ejemplo: "FAT32")          |
| 0x40   | -      | -                                        | Firma (0x55AA)                                  |

FAT32 presenta una estructura del área reservada más compleja que FAT16. Mientras FAT16 utiliza un *único sector* para esta área, FAT32 la expande a 32 *sectores*. Esta expansión se debe a la necesidad de FAT32 de manejar volúmenes de mayor tamaño y mejorar la tolerancia a fallos

### Área reservada del sistema de archivos FAT16

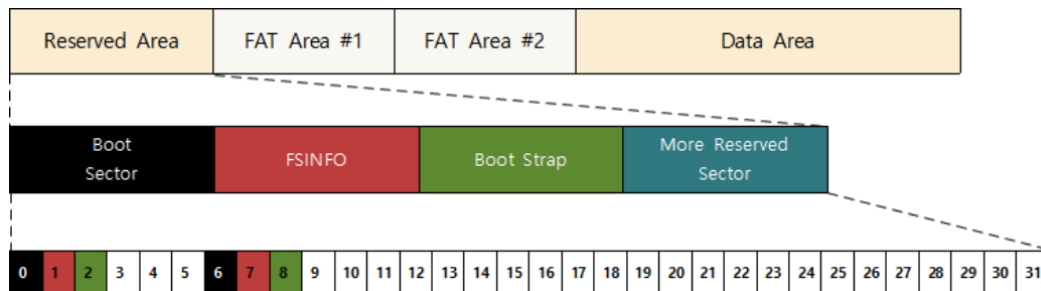


En FAT32, los 32 sectores se distribuyen de la siguiente manera:

- Sectores 0 y 6: Sectores de arranque.

- Sectores 1 y 7: Información FSINFO (estado del sistema de archivos).
- Sectores 2 y 8: Boot Strap (código de carga inicial).

#### Área reservada del sistema de archivos FAT16



Esta estructura más compleja permite a FAT32 gestionar de manera más eficiente grandes volúmenes de almacenamiento en comparación con FAT16

#### Ejemplo de lectura del Boot Sector en C

```
fseek(disk, lba_start * 512, SEEK_SET);
fread(&boot_sector, sizeof(boot_sector), 1, disk);
printf("Sistema de archivos: %.8s\n", boot_sector.fs_type);
printf("Etiqueta de volumen: %.11s\n", boot_sector.volume_label);
printf("Tamaño de clúster: %d sectores\n", boot_sector.sectors_per_cluster);
```

#### File Allocation Table (FAT1 y FAT2)

La File Allocation Table (FAT) es una estructura de datos que mantiene un mapa de asignación de clústeres en discos con sistemas FAT12, FAT16 y FAT32.

*Cada entrada en la FAT corresponde a un clúster en el disco y almacena información sobre su estado: si está libre, ocupado, reservado o dañado, o si representa el fin de la cadena de clústeres de un archivo.*

Las entradas de la FAT pueden ser de *12 bits* (FAT12), *16 bits* (FAT16) o *32 bits* (FAT32).

| Valor (Hex)             | Significado                                   | FAT12 | FAT16 | FAT32 |
|-------------------------|-----------------------------------------------|-------|-------|-------|
| 0x000                   | Clúster libre.                                | ✓     | ✓     | ✓     |
| 0xFF0–0xFF6             | Reservado para uso interno del sistema.       | ✓     | ✓     | ✓     |
| 0xFF7                   | Clúster dañado (bad cluster). No utilizable.  | ✓     | ✓     | ✓     |
| 0xFF8–0xFFFF            | Fin de la cadena de clústeres (End of Chain). | ✓     | ✓     | —     |
| 0xFFFF                  | Fin de la cadena de clústeres (End of Chain). | —     | ✓     | —     |
| 0xFFFFFFFF8–0xFFFFFFFFF | Fin de la cadena de clústeres en FAT32.       | —     | —     | ✓     |

*(Basado en Microsoft FAT32 Specification, 2000)*

Para mayor seguridad y redundancia, la FAT se mantiene en dos copias idénticas: FAT1 y FAT2. En condiciones normales, FAT1 es la principal y FAT2 funciona como respaldo. Si FAT1 se corrompe, el sistema puede reconstruir el mapa de asignación a partir de FAT2 y seguir accediendo a los archivos.

Si ambas FAT se dañan, el sistema pierde la referencia clara de las cadenas de clústeres, y la recuperación requiere técnicas forenses. Esto implica:

- Escanear el disco en busca de firmas de archivos conocidas (como JPG, DOCX o MP4).

- Reconstruir las cadenas de clústeres estimando su continuidad física en el disco.
- Emplear herramientas especializadas como *TestDisk*, *EnCase* o *FTK Imager* para recuperar archivos parciales o reconstruir directorios usando metadatos residuales.

Ejemplo de lectura de la FAT en C

```
fseek(disk, lba_start + fat_start * 512, SEEK_SET);
fread(&fat, sizeof(fat), 1, disk);
printf("Cluster 2 apunta a: %x\n", fat[2]);
```

## Directorio Raíz

El Directorio Raíz contiene entradas que describen cada archivo y subdirectorio en el nivel más alto del sistema de archivos. Cada entrada del directorio raíz incluye información como el nombre del archivo, la extensión, la fecha y hora de creación, los atributos del archivo, y el número del primer clúster del archivo en la FAT. En sistemas FAT12 y FAT16 usan una estructura similar, el Directorio Raíz tiene un tamaño fijo, mientras que en FAT32 introduce algunas diferencias importantes, como el uso de una ubicación específica para el directorio raíz y valores adicionales para manejar archivos largos (LFN)

| Offset | Tamaño  | Descripción                                        |
|--------|---------|----------------------------------------------------|
| 0x00   | 8 bytes | Nombre del archivo (en formato 8.3)                |
| 0x08   | 3 bytes | Extensión del archivo                              |
| 0x0B   | 1 byte  | Atributos (oculto, sistema, etc.)                  |
| 0x0C   | 1 byte  | Reservado (para Windows NT)                        |
| 0x0D   | 1 byte  | Hora de creación (milisegundos)                    |
| 0x0E   | 2 bytes | Hora de creación                                   |
| 0x10   | 2 bytes | Fecha de creación                                  |
| 0x12   | 2 bytes | Última fecha de acceso                             |
| 0x14   | 2 bytes | Alto del número de clúster inicial (solo en FAT32) |
| 0x16   | 2 bytes | Hora de última modificación                        |
| 0x18   | 2 bytes | Fecha de última modificación                       |
| 0x1A   | 2 bytes | Bajo del número de clúster inicial                 |
| 0x1C   | 4 bytes | Tamaño del archivo (en bytes)                      |

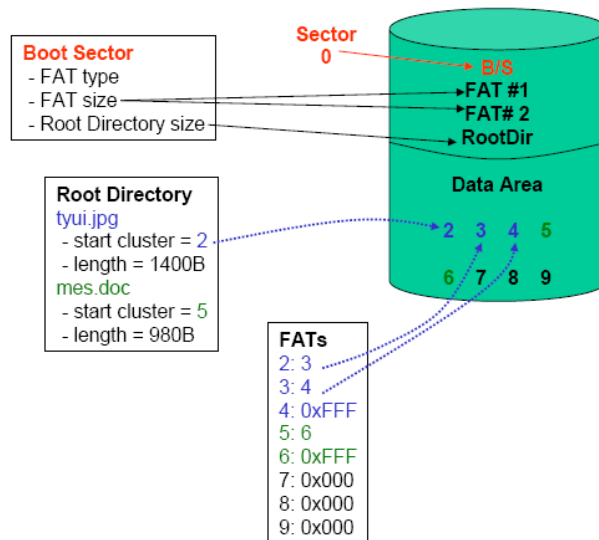
Ejemplo de lectura de la FAT en C

```
fread(&entry, sizeof(entry), 1, disk);
if (entry.name[0] != 0x00 && entry.name[0] != 0xE5) {
    printf("Archivo: %.8s.%.3s\n", entry.name, entry.ext);
}
```

## Área de Datos

Es la parte del disco donde se almacenan los datos reales de los archivos. Los datos se organizan en clústeres, y cada archivo puede ocupar uno o más clústeres.

La FAT utiliza el Área de Datos para leer y escribir la información de los archivos, localizando los clústeres correspondientes y siguiendo la cadena de clústeres para archivos que ocupan múltiples bloques.



Vamos a desglosar el siguiente ejemplo archivo tyui.jpg paso a paso para entender cómo funciona el sistema de archivos FAT al guardar archivos.

- Componentes: Root Sector, Root Directory, FAT y Data Area
- Cluster inicial: 2
- Tamaño: 1400 bytes (B).

*Paso 1. Calcular cuántos clusters necesita?:* Supongamos que cada cluster tiene un tamaño de 512 bytes.  $\Rightarrow 1400 \text{ B} / 512 \text{ B por cluster} \approx 2.73 \text{ clusters}$ . Se necesitan 3 clusters (se redondea hacia arriba).

*Paso 2. Asignación en la FAT:* La FAT gestiona la secuencia de clusters:

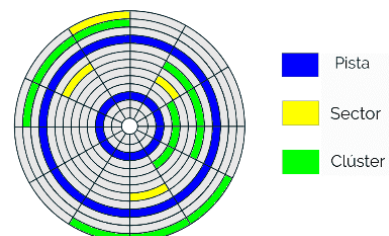
- Cluster 2: Apunta al cluster 3 (siguiente en la cadena).
- Cluster 3: Apunta al cluster 4.
- Cluster 4: Contiene 0xFFF (marca el fin del archivo en FAT12/16).
- Cadena de clusters: 2, 3, 4 (3 clusters en total).

*Paso 3. Almacenamiento físico:* Los datos de tyui.jpg se guardan en los clusters 2, 3 y 4 de la Data Area:

- Cluster 2  $\rightarrow$  Datos del archivo (primeros 512 B)
- Cluster 3  $\rightarrow$  Datos del archivo (siguientes 512 B)
- Cluster 4  $\rightarrow$  Datos restantes (376 B)

### 1.3. Conceptos Clave de Almacenamiento

**Sectores:** Un sector es la unidad mínima de almacenamiento en un disco duro o dispositivo similar, normalmente de 512 bytes. Los sectores son las divisiones físicas más pequeñas en el disco.



Los sectores son los bloques básicos de datos en los que se divide una pista en un disco, y son la unidad básica de acceso y almacenamiento de datos en un dispositivo.

**Bloques:** Un bloque es una unidad de almacenamiento compuesta por uno o más sectores. En el contexto de sistemas de archivos, los bloques son la unidad de asignación que el sistema operativo utiliza para gestionar el almacenamiento.

Los bloques agrupan sectores y se utilizan para asignar espacio en disco a los archivos de manera más eficiente.

**Clústeres:** Un clúster es un conjunto de uno o más bloques contiguos que representan la unidad mínima de almacenamiento que se puede asignar a un archivo en un sistema de archivos FAT.

Los clústeres son la unidad mínima de espacio que se puede asignar a un archivo. La FAT mantiene una relación uno a uno con los clústeres, rastreando su estado y ubicación en el disco

## Sistemas de Archivos NTFS

NTFS (New Technology File System) es un sistema de archivos desarrollado por Microsoft para la familia Windows NT. Su diseño buscó superar las limitaciones de FAT mediante una arquitectura más robusta, seguridad avanzada, tolerancia a fallos y mejor aprovechamiento del espacio en disco. Actualmente es el estándar en entornos críticos y empresariales por su capacidad de combinar eficiencia operativa con mecanismos de protección y recuperación.

**Tabla Maestra de Archivos (MFT):** NTFS utiliza como núcleo la Master File Table (MFT), donde cada archivo o directorio tiene un registro único con información completa y punteros a datos. NTFS incorpora mecanismos como listas de control de acceso (ACLs), cifrado nativo (EFS), journaling de metadatos y soporte para atributos extendidos, lo que lo convierte en un sistema robusto y adecuado para entornos críticos

### Estructura de un registro de MFT

Un registro típico de la MFT tiene un encabezado y tributos principales:

*Encabezado:* Identificador del tipo de archivo (archivo normal, directorio, metadatos del sistema). Estado del registro: activo, borrado o inactivo. Y secuencia de actualización para proteger la consistencia interna ante fallos.

*Standard Information:* Contiene información esencial sobre el archivo o directorio, como fechas de creación, modificación, acceso, flags de archivo (oculto, sistema, de solo lectura), y referencias a las versiones de atributos.

*File Name:* Almacena el nombre lógico del archivo, el namespace (Win32/Unicode) y la ubicación lógica dentro del árbol de directorios.

*Data Attribute:* Punteros a los bloques de datos que contienen el contenido real del archivo. Puede ser residente (contenido pequeño dentro del registro MFT) o no residente (datos en clusters externos).

*Security Descriptor:* Incluye ACLs (Access Control Lists), permisos, auditoría y cifrado, proporcionando control de acceso granular.

## Características de NTFS

### Recuperación ante fallos

NTFS incorpora un sistema de *journaling de metadatos* basado en la técnica de *Write-Ahead Logging*. Este mecanismo asegura que, ante un fallo o corte de energía, el sistema de archivos

pueda regresar a un estado consistente. La MFT facilita recuperación de archivos borrados y gestión de archivos fragmentados

### **Seguridad**

El sistema implementa ACLs (Access Control Lists) para establecer permisos granulares sobre archivos y directorios, junto con cifrado nativo mediante EFS (Encrypting File System). A ello se suman funciones de auditoría que permiten rastrear accesos y modificaciones. Estas características convierten a NTFS en una solución confiable para entornos donde la seguridad y la trazabilidad son prioritarias.

### **Metadatos**

Además, NTFS soporta múltiples atributos, archivos residentes y no residentes, así como flujos alternativos de datos. Incluye permisos, timestamp, punteros a bloques, atributos extendidos, flujos alternativos de datos y funcionalidades de auditoría, lo que lo convierte en un sistema mucho más seguro y confiable en entornos empresariales.

### **Relevancia forense de la MFT**

La MFT es esencial en análisis forense digital. Permite reconstruir directorios, recuperar archivos borrados (ya que los registros persisten hasta ser sobrescritos) y analizar la actividad de usuarios mediante timestamps..

Cuando un archivo es eliminado, su registro MFT no se borra inmediatamente; los metadatos persisten hasta que se sobrescriben, facilitando la recuperación forense.

Los investigadores pueden analizar los timestamps para identificar patrones de acceso, modificación y creación de archivos, lo cual es de gran utilidad en auditorías de seguridad. Además, la MFT ayuda a detectar y analizar la fragmentación, ya que los punteros a clusters dispersos reflejan la manera en que los datos de un archivo se distribuyen en el disco, información fundamental en la recuperación de archivos fragmentados

## **Ejercicios**

1. *Dado un sistema de archivos FAT16 con un tamaño de bloque de 512 bytes, un tamaño de clúster de 8 bloques y una tabla FAT con 1024 entradas.*

*a. Determinar el tamaño máximo de un archivo que puede ser almacenado en este sistema de archivos. Justificar de manera detallada el cálculo realizado.*

2. *En un sistema de archivos FAT16 con un tamaño de bloque de 512 bytes y clúster de 8 bloques.*

*a. Calcular el número de entradas necesarias en la tabla FAT para direccionar un espacio de almacenamiento total de 1 GB.*

*b. Calcular el tamaño total de la FAT*



## 2. Origen y Evolución de UNIX

Desde una perspectiva histórica, los sistemas de archivos UNIX han evolucionado para satisfacer las crecientes demandas de rendimiento, confiabilidad y escalabilidad. El diseño inicial de **UFS** (UNIX File System) se basaba en una estructura jerárquica con inodos y bloques de datos, pero a medida que los sistemas crecían en complejidad, surgió la necesidad de soluciones más eficientes como **Ext** (Extended File System). La evolución de Ext ha introducido mejoras clave como **journaling** (para una rápida recuperación tras fallos), el uso de **extents** (para reducir la fragmentación) y estructuras avanzadas para manejar volúmenes de datos de varios terabytes.

### 1. Origen y Evolución de Ext (Extended File System)

#### UFS, EXT2, EXT3, EXT4 sistema de archivos UNIX (UFS)

**UFS** se originó en 1979 en los laboratorios Bell, basado en el sistema de archivos original de UNIX. La estructura de UFS está compuesta por inodos para gestionar metadatos y bloques de datos para almacenar el contenido de los archivos.

**EXT** fue el primer sistema de archivos diseñado para Linux, basado en UFS, pero con mejoras en la asignación de espacio mediante bitmaps. A lo largo de las diferentes versiones (Ext2, Ext3 y Ext4), se introdujeron mejoras como el uso de journaling y extents.

**EXT2** Primera revisión importante de Ext. Mejoró la gestión con bitmaps de bloques e inodos, pero sin journaling.

**EXT3** incorporación de journaling (recuperación rápida tras un fallo). Compatibilidad retroactiva con Ext2 y mejor resistencia a fallos y recuperación más rápida.

**EXT4** utiliza extents para mejorar la gestión de fragmentación, tamaños de archivo y volumen extendidos (hasta 16 TB por archivo), y mejoras en rendimiento y fiabilidad. Es el sistema de archivos predeterminado en la mayoría de las distribuciones Linux.

Tabla de Evolución de Sistemas de Archivos en Linux

| Sistema de Archivos | Año de Introducción | Capacidad Máxima de Archivo | Tamaño Máximo Volumen | Tamaño de Bloque | Journaling | Sistema de Asignación | Compatibilidad con versiones anteriores |
|---------------------|---------------------|-----------------------------|-----------------------|------------------|------------|-----------------------|-----------------------------------------|
| <b>Ext</b>          | 1992                | 2 GB                        | 2 GB                  | 1 KB             | No         | List enlazadas        | No                                      |
| <b>Ext2</b>         | 1993                | 2 TB                        | 32 TB                 | 1 KB – 4 KB      | No         | Bitmaps               | No                                      |
| <b>Ext3</b>         | 2001                | 2 TB                        | 32 TB                 | 1 KB – 4 KB      | Si         | Bitmaps               | Si (con Ext2)                           |
| <b>Ext4</b>         | 2008                | 16 TB                       | 1 EB                  | 1 KB – 64 KB     | Si         | Extents               | Sí (con Ext3)                           |
| <b>XFS</b>          | 1993                | 8 EB                        | 8 EB                  | 4 KB – 64 KB     | Si         | B-Trees               | No                                      |
| <b>Btrfs</b>        | 2007                | 16 EB                       | 16 EB                 | 4 KB             | Si         | Extents               | No                                      |
| <b>ZFS</b>          | 2005                | 16 EB                       | 256 ZB                | 4 KB – 128KB     | Si         | B-Trees               | No                                      |

#### Estructura del Sistema de Archivos Ext

La estructura de sistema de archivos Ext (Extended File System) es compleja y su diseño está pensado para ofrecer un manejo eficiente de archivos, capacidad de recuperación tras fallos y escalabilidad para soportar discos de gran tamaño. La estructura interna de Ext está organizada de manera jerárquica y lógica para permitir una gestión eficiente de los datos y los metadatos. En términos generales, el sistema de archivos Ext está compuesto por tres elementos fundamentales: Superbloque, Grupos de bloques y Inodos

Cada uno de estos componentes tiene una función específica y está interrelacionado para garantizar la integridad y eficiencia en el acceso a los datos. A continuación, se detalla la estructura completa de Ext y el funcionamiento de cada componente

## MBR/GPT

Al igual que en FAT, el MBR (Master Boot Record) o el GPT (GUID Partition Table) se encuentran fuera del sistema de archivos Ext. Este MBR o GPT se encuentra en el primer sector del disco y tiene la responsabilidad de definir cómo se distribuye el espacio del disco en particiones. Mientras que FAT y Ext utilizan el espacio dentro de las particiones para almacenar sus datos, el MBR/GPT organiza la partición del disco, indicando su ubicación, tamaño y otros metadatos importantes.

Aunque el MBR/GPT no forma parte directamente del sistema de archivos, es importante entender su funcionamiento porque establece la partición en la que se montará el sistema de archivos Ext. En el caso de Ext, dentro de una partición, se encuentra el *superbloque*, que es la estructura clave para que el sistema de archivos se cargue y funcione correctamente.

Supongamos que un disco de 1 TB tiene tres particiones de 300 GB, 300 GB y 400 GB. El MBR contendrá la información sobre estas particiones y la dirección de inicio de cada una. Sin embargo, cada partición tendrá su propia estructura Ext (superbloque, bitmaps, inodos y bloques de datos, etc).

### 1. Superbloque (Superblock)

El *superbloque* es el componente central que contiene la información global sobre el sistema de archivos Ext. Su función es proporcionar una vista general de la estructura y configuración del sistema de archivos. Si el superbloque está dañado, el sistema de archivos puede volverse inaccesible o incoherente.

El *superbloque* se ubica en el *bloque 1* (segundo bloque del disco) y se replica en cada grupo de bloques para redundancia. Para aumentar la redundancia y mejorar la capacidad de recuperación, copias del *superbloque* se almacenan en cada grupo de bloques.

En sistemas Ext2 y Ext3, las copias del superbloque solo se almacenan en ciertos grupos de bloques. En Ext4, las copias del superbloque se almacenan de manera más distribuida (opción *sparse\_super*).

El superbloque contiene metadatos fundamentales

| Campo               | Descripción                                                  | Ejemplo                |
|---------------------|--------------------------------------------------------------|------------------------|
| s_inodes_count      | Número total de inodos                                       | 1,000,000 inodos       |
| s_blocks_count      | Número total de bloques                                      | 1,000,000 bloques      |
| s_free_blocks_count | Número de bloques libres                                     | 250,000 bloques        |
| s_free_inodes_count | Número de inodos libres                                      | 800,000 inodos         |
| s_log_block_size    | Tamaño del bloque (como potencia de 2)                       | 4 KB                   |
| s_mtime             | Última vez que el sistema de archivos fue montado            | 1678459215 (timestamp) |
| s_magic             | Número mágico que identifica el sistema de archivos (0xEF53) | 0xEF53                 |
| s_state             | Estado del sistema de archivos                               | 1 (montado)            |

Si el tamaño del bloque es de 4 KB y el número total de bloques es de 1,000,000, el tamaño total del sistema de archivos será:

Tamaño total= tamaño de bloque x número de bloques

$$4\text{KB} \times 1,000,000 = 4,096,000,000\text{KB} = 3.81\text{GB}$$

Recuperación ante fallos: Si el superbloque principal se corrompe, el sistema usa las copias almacenadas en otros grupos de bloques (Tanenbaum, 2015).

## 2. Grupos de Bloques (Block Groups)

El sistema Ext está dividido en *grupos de bloques* para mejorar la eficiencia y la redundancia. Cada grupo funciona como una unidad autosuficiente que contiene sus propias estructuras de administración de datos. La división en grupos de bloques permite minimizar la fragmentación y facilita la recuperación tras fallos.

| Componente        | Descripción                                                             | Función Clave                                                                  |
|-------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Bitmap de Bloques | Secuencia de bits que indica si un bloque está libre (0) u ocupado (1). | Optimiza la asignación rápida de bloques libres.                               |
| Bitmap de Inodos  | Secuencia de bits que indica si un inodo está libre (0) u ocupado (1).  | Facilita la gestión eficiente de inodos.                                       |
| Tabla de Inodos   | Array de estructuras de inodos (256 bytes cada una en ext4).            | Almacena metadatos de archivos/directorios (permisos, tamaño, punteros, etc.). |
| Bloques de Datos  | Bloques que almacenan el contenido real de archivos y directorios.      | Espacio físico para datos, gestionado por los inodos.                          |

### Bitmap de Bloques Libres

El *bitmap de bloques libres* es una estructura de datos que indica el estado de cada bloque de datos en el sistema de archivos Ext. Su función principal es rastrear qué bloques están disponibles para ser utilizados y cuáles están ocupados.

El bitmap de bloques libres está organizado como una secuencia de bits, donde cada bit representa un *bloque individual*.

Si el valor del bit es:

- **0** → El bloque está *libre* y puede ser asignado a un archivo o directorio.
- **1** → El bloque está *ocupado* y contiene datos o metadatos activos

Ejemplo, si el bitmap de bloques es: 1111000011110000

Esto indica que: los primeros 4 bloques están ocupados y los siguientes 4 bloques están libres.

### Bitmap de Inodos Libres

El *bitmap de inodos libres* funciona de manera similar al bitmap de bloques libres, pero en lugar de rastrear bloques de datos, rastrea la disponibilidad de *inodos* (estructuras que almacenan metadatos de archivos).

Cada bit en el bitmap de inodos representa un *inodo individual* dentro del grupo de bloques.

Si el valor del bit es:

- **0** → El inodo está **libre** y puede ser asignado a un nuevo archivo o directorio.
- **1** → El inodo está **ocupado** y está siendo utilizado por un archivo o directorio.

Ejemplo: Si el grupo de bloques tiene 8 inodos, el bitmap de inodos podría verse así: 11010011

Esto indica que: Los inodos **0** y **1** están ocupados, el inodo **2** está libre y puede ser usado para un nuevo archivo, los inodos **3** y **7** están ocupados, Los inodos **4** y **5** están libres.

### Tabla de Inodos

La *tabla de inodos* es una estructura ordenada que almacena todos los *inodos individuales* dentro de un grupo de bloques. Cada grupo de bloques en el sistema de archivos Ext contiene su propia tabla de inodos, lo que permite que la gestión y búsqueda de archivos sea rápida y eficiente.

### Ubicación y Organización de la Tabla de Inodos:

La tabla de inodos está ubicada dentro de cada *grupo de bloques*. Cada grupo de bloques tiene su propia tabla de inodos independiente. El número total de inodos está definido en el *superbloque* al momento de formatear el sistema de archivos y se distribuye entre todos los grupos de bloques.

La tabla de inodos es simplemente un área reservada dentro de cada grupo de bloques que contiene una *secuencia de inodos numerados* (índices de inodos).

El tamaño de cada inodo en la tabla de inodos es *fijo* y depende de la versión del sistema de archivos Ext:

- **128 bytes** en Ext2.
- **256 bytes** en Ext3 y Ext4.

Si el sistema de archivos Ext4 tiene 1 millón de inodos y cada inodo ocupa 256 bytes, el tamaño total de la tabla de inodos será:

Tamaño de la tabla de inodos = Número total de inodos x Tamaño de cada inodo

$1,000,000 \times 256 = 256,000,000 \text{ bytes} = 256 \text{ MB}$

### Bloques de datos

Los bloques de datos contienen el contenido real de los archivos y directorios. Cuando un archivo o directorio es creado o modificado, su contenido se almacena en estos bloques. Los bloques de datos se gestionan a través de los *inodos*, que contienen los punteros a los bloques específicos.

Los bloques de datos pueden tener diferentes tamaños, dependiendo de la configuración del sistema de archivos, aunque en la mayoría de las implementaciones modernas, como en Ext4, el tamaño de los bloques es típicamente de 4 KB. Este tamaño puede ser configurable al momento de crear el sistema de archivos, dependiendo de las necesidades de almacenamiento.

Si el tamaño de bloque es de 4 KB y un archivo mide 10 KB, se necesitarán 3 bloques: Los dos primeros bloques contendrán 4 KB de datos cada uno. El tercer bloque contendrá 2 KB de datos (dejando 2 KB libres).

### 3. Inodos (Metadatos)

Un **inodo** es una estructura de datos que almacena los **metadatos** asociados a un archivo o directorio. Cada archivo y directorio en el sistema de archivos Ext tiene un inodo único que almacena toda la información relevante sobre ese archivo, excepto su nombre (que está almacenado en la estructura de directorios).

Los inodos permiten al sistema de archivos rastrear los datos almacenados en los bloques de datos. Cada inodo tiene un número único dentro del sistema de archivos (número de inodo).

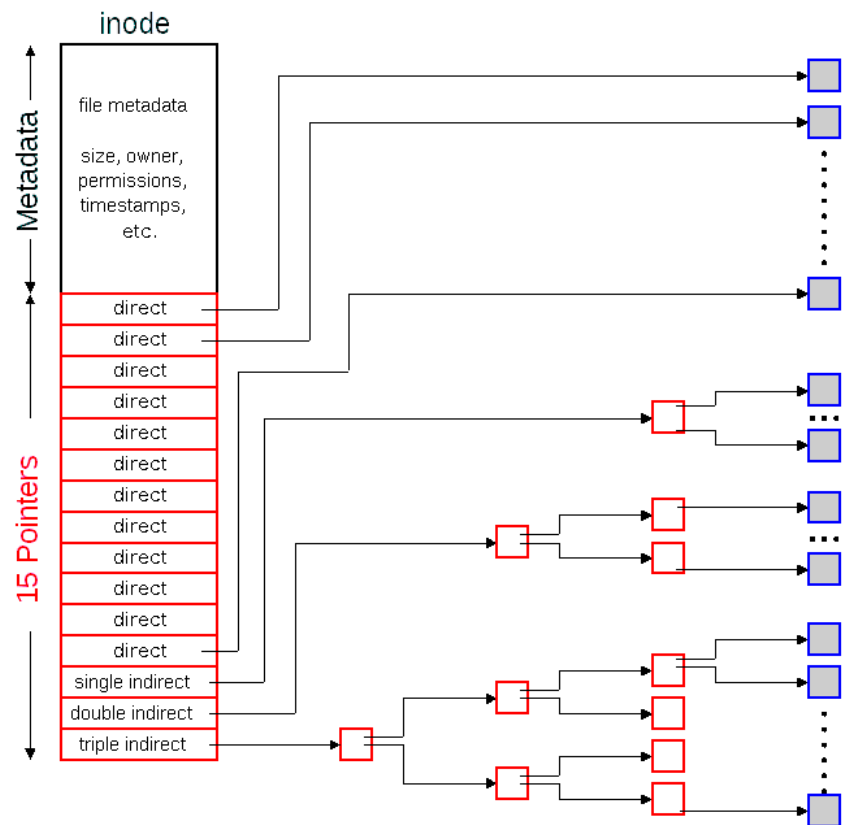
Cada inodo contiene información sobre el archivo o directorio, que incluye:

| Campo  | Descripción                                                              |
|--------|--------------------------------------------------------------------------|
| i_mode | Permisos y tipo de archivo (regular, directorio, enlace simbólico, etc.) |

|               |                                                         |
|---------------|---------------------------------------------------------|
| i_uid         | ID del propietario del archivo                          |
| i_gid         | ID del grupo asociado al archivo                        |
| i_size        | Tamaño del archivo en bytes                             |
| i_blocks      | Número de bloques asignados                             |
| i_atime       | Fecha y hora del último acceso                          |
| i_ctime       | Fecha y hora de la última modificación de los metadatos |
| i_mtime       | Fecha y hora de la última modificación del contenido    |
| i_links_count | Número de enlaces duros al archivo                      |
| i_flags       | Indicadores del estado del archivo                      |
| i_generation  | Número de versión del archivo                           |

El tipo de archivo se define mediante el campo i\_mode del inodo

Los inodos almacenan las ubicaciones de los bloques de datos mediante un conjunto de punteros: *Punteros Directos e Indirectos*



En los sistemas de archivos *Ext2/Ext3* , un inodo estándar contiene:

| Tipo de Puntero            | Función                                                                     |
|----------------------------|-----------------------------------------------------------------------------|
| 12 Punteros directos       | Apuntan directamente a los bloques de datos.                                |
| 1 Puntero indirecto simple | Apunta a un bloque que contiene direcciones de bloques de datos.            |
| 1 Puntero indirecto doble  | Apunta a un bloque que contiene direcciones de bloques de punteros simples. |
| 1 Puntero indirecto triple | Apunta a un bloque que contiene direcciones de bloques de punteros dobles.  |

Estos punteros directos e indirectos están diseñados para facilitar el acceso a los bloques de datos de un archivo, y la cantidad de punteros está limitada por la capacidad del inodo (normalmente 128 o 256 bytes).

### 3. Enlaces de archivos: Hard Links y Soft Links

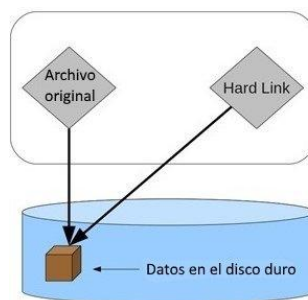
En los sistemas operativos modernos, especialmente en los sistemas de archivos basados en UNIX/Linux y NTFS, los archivos pueden estar asociados a múltiples nombres o ubicaciones dentro del sistema de archivos a través de un mecanismo denominado enlaces (links). Estos enlaces permiten que el mismo archivo o directorio sea accesible desde múltiples rutas, lo que ofrece ventajas tanto en términos de organización como de redundancia y referenciación sin la necesidad de duplicar el contenido del archivo.

Existen dos tipos principales de enlaces: Hard Links (enlaces duros) y Soft Links (enlaces simbólicos o symlinks). Cada tipo de enlace tiene características y aplicaciones distintas, y su funcionamiento depende en gran medida de la implementación en el sistema de archivos subyacente. A continuación, se analiza en detalle el funcionamiento de ambos tipos de enlaces.

### Hard Links (Enlaces duros)

Un enlace duro es una referencia directa a un mismo archivo físico en el disco. En los sistemas de archivos tipo UNIX/Linux (como EXT4), cada archivo está representado por un inodo (estructura que almacena información sobre el archivo, como permisos, propietario, tamaño y ubicación de bloques de datos).

Cuando se crea un enlace duro, se agrega una nueva entrada en el directorio que apunta al mismo inodo del archivo original. Es decir, el enlace duro no tiene una ruta independiente, sino que comparte la misma referencia de inodo que el archivo original. Por lo tanto, múltiples nombres de archivo pueden referirse al mismo archivo físico en disco.



En un sistema de archivos basado en inodos como EXT4 o NTFS, cuando se crea un enlace duro, el *contador de enlaces (link count)* del inodo se incrementa en uno. Este contador refleja cuántos nombres de archivo están asociados con ese inodo. Los archivos no se eliminan del sistema hasta que el contador de enlaces llegue a cero, es decir, cuando todos los enlaces duros asociados a ese inodo han sido eliminados.

Para crear un enlace duro en Linux, se utiliza el comando *ln*. A continuación, se muestra cómo crear un enlace duro:

```
# Crear un hard link
ln archivo_original enlace_duro

# Verificar los enlaces asociados
ls -li archivo_original enlace_duro
```

Este comando muestra el inodo y los enlaces asociados a ambos archivos. Como se puede ver, *el inodo será el mismo para ambos archivos*.

En NTFS, Windows también soporta los enlaces duros, aunque no de la misma manera que los sistemas basados en UNIX. NTFS maneja los enlaces duros a través de su estructura de metadatos. Para crear un enlace duro en Windows, se utiliza el comando *fsutil*:

```
# Crear un hard link en NTFS
```

```
fsutil hardlink create enlace_duro archivo_original
```

El sistema de archivos NTFS gestiona los hard links mediante las estructuras de metadatos.

**EXT4 (Linux):** Soporta hard links y mantiene el contador de enlaces en el inodo.

**NTFS (Windows):** Soporta hard links mediante entradas de directorio vinculadas a la misma referencia de datos.

### Características y Restricciones

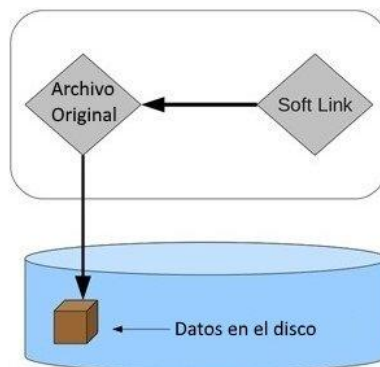
- **Contador de Enlaces:** El contador de enlaces en el inodo refleja cuántos nombres están asociados al archivo. Mientras haya al menos un enlace, el archivo no se elimina.
- **No pueden cruzar sistemas de archivos:** No es posible crear un enlace duro que apunte a un archivo en otro sistema de archivos.
- **No se pueden crear enlaces duros para directorios:** Para evitar la creación de ciclos infinitos y mantener la jerarquía de directorios sin problemas, no se permiten enlaces duros a directorios.

### Seguridad y Permisos

Los permisos asociados a un enlace duro son los mismos que los del archivo original. Si se modifican los permisos o el propietario de uno de los enlaces, estos cambios se aplican a todos los enlaces duros asociados al mismo inodo. Además, los usuarios sólo pueden crear enlaces duros para archivos sobre los que tengan permisos de *lectura y escritura*.

### Soft Links (Enlaces simbólicos)

Un enlace simbólico, o symlink, es una referencia indirecta a un archivo o directorio. A diferencia de los enlaces duros, los symlinks no hacen referencia al inodo del archivo original. En lugar de ello, almacenan una ruta (relativa o absoluta) que apunta al archivo o directorio de destino.



Cuando un symlink es accedido, el sistema operativo resuelve la ruta del enlace simbólico y redirige la operación hacia el archivo o directorio de destino. En caso de que el archivo de destino se elimine, el symlink se convierte en un enlace roto (dead link), es decir, deja de ser válido.

Para crear un enlace simbólico en Linux, se utiliza el comando *ln* con la opción *-s*:

```
# Crear un symlink
```

```
ln -s archivo_original enlace_simbolico
```

```
# Verificar enlaces simbólicos
```

```
ls -l archivo_original enlace_simbolico
```

En NTFS, los symlinks son soportados, pero de manera diferente a los enlaces duros. En Windows, se puede crear un symlink utilizando el comando mklink:

```
# Crear un symlink en NTFS
```

```
cmd /c mklink enlace_simbolico archivo_original
```

**EXT4:** Los symlinks se almacenan como inodos independientes con un tipo especial.

**NTFS:** Soporta symlinks mediante estructuras específicas del sistema de archivos.

### Características y Restricciones

- **Enlaces Simbólicos Cruzados:** A diferencia de los enlaces duros, los symlinks pueden cruzar sistemas de archivos diferentes y apuntar a archivos fuera del sistema de archivos local.
- **Enlaces a Directorios:** Los symlinks pueden apuntar tanto a archivos como a directorios, lo que los hace más versátiles que los enlaces duros.

### Seguridad y Permisos

A diferencia de los enlaces duros, los permisos de los symlinks son independientes del archivo original. Sin embargo, el acceso al archivo de destino siempre está condicionado a los permisos de dicho archivo. Además, los symlinks pueden ser utilizados para redirigir operaciones privilegiadas a archivos críticos, lo que los hace susceptibles a ataques de *escalamiento de privilegios* si no se gestionan adecuadamente.

### Diferencias clave entre enlaces duros y simbólicos

| Característica                                   | Hard Link                   | Soft Link                    |
|--------------------------------------------------|-----------------------------|------------------------------|
| Apunta a                                         | Inodo                       | Ruta de archivo              |
| Relación con el inodo                            | Mismo inodo                 | Inodo independiente          |
| Puede cruzar sistemas de archivos                | No                          | Sí                           |
| Afectado por la eliminación del archivo original | No                          | Sí                           |
| Soporte en sistemas de archivos                  | EXT4, NTFS                  | EXT4, NTFS, FAT32 (limitado) |
| Permisos                                         | Hereda del archivo original | Independiente                |

### Errores al eliminar o manipular enlaces

Cuando se elimina un enlace duro utilizando el comando rm, el archivo no se eliminará hasta que todos los enlaces duros asociados al mismo inodo sean eliminados.

Si intentas acceder o eliminar un *symlink* cuyo archivo de destino fue eliminado, el sistema mostrará un error del tipo:

```
cat: enlace_simbolico: No such file or directory
```



```
rm: cannot remove 'enlace_simbolico': No such file or directory
```

En el caso de hard links, si todos los enlaces son eliminados, el sistema liberará el espacio de disco automáticamente.

Si intentas crear un hard link a un directorio o entre sistemas de archivos distintos, el sistema mostrará un error similar a:

```
ln: creating hard link 'destino' => 'origen': Invalid cross-device link
```

En el caso de symlinks, si apuntas a un archivo fuera del sistema de archivos o el destino es inaccesible, se generará un error de referencia o archivo no encontrado.

## Seguridad y Gestión de Enlaces

Uno de los riesgos más significativos de los symlinks es su potencial para ser explotados en ataques **TOCTOU** (Time-of-Check to Time-of-Use). Los symlinks permiten que un archivo o directorio se redirija a una ruta distinta, y si no se gestionan adecuadamente, pueden usarse para eludir restricciones de acceso y manipular archivos en sistemas de archivos. Esto ocurre cuando un atacante crea un symlink que apunta a un archivo o directorio crítico del sistema y, a través de un desfase temporal, puede modificar el archivo al que apunta el enlace.

### Ataque TOCTOU

Un ataque TOCTOU (Time-of-Check to Time-of-Use) ocurre cuando un atacante manipula un symlink para redirigir una operación a un archivo distinto al que se había verificado inicialmente. Este tipo de ataque puede ocurrir si un programa realiza una verificación de permisos o acceso a un archivo, pero antes de que se use, el atacante cambia el archivo al que apunta el symlink.

Para evitar los ataques TOCTOU, se recomienda realizar verificaciones adicionales en las rutas de los symlinks y asegurarse de que los permisos sean revisados de manera exhaustiva antes de realizar cualquier operación sobre el archivo. Además, se pueden implementar políticas de seguridad más estrictas sobre quién puede crear symlinks y en qué directorios.

## Ejercicio

Consideremos un ejemplo práctico para ilustrar la diferencia entre enlaces físicos y simbólicos:

- Tiene un archivo llamado `archivo.c` en `directorio1`.
- Se realizan los siguientes pasos:

1. Crear un enlace duro (`j.txt`) y otro enlace duro (`l.txt`) a `archivo.c` en `directorio1`:

```
ln archivo.c j.txt
ln archivo.c l.txt
```

Ambos (`j.txt` y `l.txt`) compartirán el mismo inodo que `archivo.c`.

2. Crear un enlace simbólico (`i.txt`, `k.txt`, `m.txt`) a `archivo.c` en `directorio1`:

```
ln -s archivo.c i.txt
ln -s archivo.c k.txt
ln -s archivo.c m.txt
```

Cada uno de estos enlaces simbólicos tiene su propio inodo y contiene la ruta a archivo.c.

3. Eliminar archivo.c y j.txt:

```
rm archivo.c
rm j.txt
```

Aunque se elimina el archivo original (archivo.c), todavía puedes acceder al contenido a través del enlace duro l.txt. Sin embargo, los enlaces simbólicos (i.txt, k.txt, m.txt) ahora estarán rotos, ya que la ruta a archivo.c ya no es válido.

## Ejercicios

1. Se tiene un filesystem de tipo ext2 con bloques de 4 KB, punteros de 4 bytes, 5 puntero directos. 4 punteros indirectos y 2 puntero con doble indirección.

Averiguar:

- Tamaño máximo de archivo
- ¿Qué byte comienza en el puntero de doble indirección?

2. Imaginemos una partición de disco con bloques de 4 KB en un sistema de archivos basado en inodos. Cada inodo cuenta con cinco punteros directos, tres indirectos simples, dos indirectos dobles y un indirecto triple, con cada puntero representando una referencia a un bloque de datos. Si para referenciar a un bloque se utilizan 32 bits,

- ¿Cuál es el tamaño máximo del archivo?
- Encuentre eficientemente el bloque correspondiente al 500MB del archivo

3. Suponga que tiene un sistema xubuntu/Linux y desea utilizar Python para administrar archivos, debes realizar las siguientes tareas:

- Crear un hard link del archivo.py llamado hard1.py
  - Crear un hard link del archivo.py llamado hard2.py
  - Crear un soft link del archivo.py llamado soft1.py
  - Crear un soft link del archivo.py llamado soft2.py
  - Crear un soft link del archivo soft1.py llamado soft3.py
  - Eliminar los archivos hard1.py, soft3.py y archivo.py
- ¿Se puede seguir accediendo al contenido que tenía archivo.py? Realice un diagrama y justifique adecuadamente su respuesta.